

BOS: Overlap-Aware List Scheduling with Checkpoint Recovery and Delay Certificates

Chen-Hong

Department of Computer Science and Information Engineering
National Central University, Taoyuan, Taiwan
leon28473787@gmail.com

Abstract—Overlapping operations can shorten nominal schedules, but interrupted execution requires dependencies to follow attained progress rather than elapsed time. We study identical-machine scheduling with bounded start–start lags and a two-layer recovery architecture. The nominal layer uses Bottleneck-Oriented Overlap Scheduling (BOS). At a disruption, completed progress is frozen and remaining work is transformed into a checkpoint DAG with material-release gates. A single blocking-chain argument proves a release-aware bound $C_{\text{rec}} \leq (2 - 1/m)\text{OPT}_{\text{rec}} + D/m$, where D is the maximum residual release. Thus $D \leq m\epsilon B$, for a valid residual lower bound B , is sufficient for a $(2 - 1/m + \epsilon)$ certificate. We distinguish an LP that minimizes makespan at a prescribed delay from an LP that maximizes tolerable arrival delay under a fixed delivery deadline. Fractional solutions are converted to feasible schedules by executable-list rounding and validated against exact integer models. Fresh experiments include 2,880 nominal instances and 432 recovery/deadline scenarios, with 528 certified small-instance oracle solutions in total. LP portfolios give limited gains over greedy recovery, while a fixed-effort random-list portfolio performs better on these small synthetic cases. The main result is a precise recovery model and auditable delay certificate, not a claim that LP rounding universally improves scheduling quality.

Index Terms—parallel-machine scheduling, bounded overlap, checkpoint recovery, linear programming, approximation bounds

I. INTRODUCTION

A short nominal production schedule is not necessarily an adequate recovery plan. A downstream operation may begin after a predecessor has produced enough intermediate material, while a later material delivery can stop an already running operation. In this setting, the useful questions include not only the nominal completion time, but also which partial work can be preserved, which operations may proceed safely, and how much additional waiting fits within a certified delivery allowance.

This paper studies these questions in a deliberately restricted, inspectable model. In normal operation, Bottleneck-Oriented Overlap Scheduling (BOS) prioritizes the longest overlap-adjusted downstream path on identical machines. At one observed disruption, the executed prefix is frozen and remaining operations are divided at physical progress checkpoints. These segments form a new DAG. An LP relaxation provides a lower bound and expected-start priorities; a work-conserving list scheduler converts those priorities to a feasible segment schedule. The best of this schedule and two greedy schedules is retained.

The distinction between elapsed time and attained progress is essential. If a predecessor starts at time zero and is suspended after two units of work, waiting until time six does not mean that six units have been produced. Simply retaining its original start-time lag would therefore permit invalid handoffs. Our recovery graph enforces attained progress, allows a paused task to resume on another identical machine, and keeps the total processing volume unchanged.

The contributions are the following. First, we give a complete single-blocking-chain proof for nonnegative bounded start lags and work-conserving list policies. Second, we specify and implement a checkpoint transformation, including already-running tasks, zero-lag edges, and internal material gates. Third, we sharpen the release-aware bound to a delay certificate: for maximum residual release D ,

$$C_{\text{rec}} \leq (2 - 1/m)\text{OPT}_{\text{rec}} + D/m.$$

Consequently, $D \leq m\epsilon B$, for any valid residual lower bound B , is sufficient for a $(2 - 1/m + \epsilon)$ ratio. Fourth, a separate deadline-constrained LP maximizes tolerable material delay, keeping that objective distinct from makespan minimization. Finally, fresh synthetic experiments compare six priorities, LP-list rules, and protected portfolios against certified small-instance optima. The certificate is priority-independent; the LP must justify its added cost through measurements rather than through an unsupported rounding claim.

A. Relation to Existing Work

Classical list-scheduling analysis bounds makespan using workload and a critical chain [1]. Generalized temporal precedence and resource-constrained project networks predate BOS [2]. We do not claim that start lags or DAG critical paths are new. Instead, we specialize to lags bounded by the predecessor processing time, where a charged prefix is guaranteed to contain an executing task. This restriction is what makes the proof below work.

HEFT was developed for heterogeneous processors [3]. Our HEFT-rank comparator uses its identical-machine, zero-communication upward-rank expression within the same overlap-aware dispatch engine. It is not the full heterogeneous HEFT algorithm. Stronger generalized project-scheduling solvers, including lazy clause generation for RCPSP/max [4], address a broader constraint setting. Here we use a transparent

time-indexed MILP as a small-instance oracle, not as a claim of state-of-the-art exact-solver performance.

The study is a synthetic algorithmic investigation; industrial applicability and the novelty of the complete combination require further benchmarking and literature assessment.

II. MODEL AND LOWER BOUNDS

Let $G = (V, E)$ be a DAG. Task $i \in V$ has processing time $p_i > 0$ and finite release date $r_i \geq 0$. There are m identical machines. Each task uses one machine, runs non-preemptively, and is assigned a start time s_i and machine $\mu_i \in \{1, \dots, m\}$.

For edge $(i, j) \in E$, let

$$0 \leq \delta_{ij} \leq p_i, \quad \ell_{ij} = p_i - \delta_{ij} \in [0, p_i]. \quad (1)$$

The overlap allowance δ_{ij} does not shorten either task; it only reduces the elapsed start separation. Because a nominal task is uninterrupted, this also equals the amount of its execution completed before a successor may start. This equivalence must be revisited after preemption (Section V). A feasible schedule satisfies

$$s_i \geq r_i \quad \forall i \in V, \quad (2)$$

$$s_j \geq s_i + \ell_{ij} \quad \forall (i, j) \in E, \quad (3)$$

and intervals assigned to one machine are disjoint:

$$[s_i, s_i + p_i) \cap [s_j, s_j + p_j) = \emptyset \quad \text{if } i \neq j \text{ and } \mu_i = \mu_j. \quad (4)$$

The objective is

$$C_{\max} = \max_{i \in V} (s_i + p_i). \quad (5)$$

Finish–start precedence is recovered by $\delta_{ij} = 0$; a zero lag is obtained by $\delta_{ij} = p_i$ and permits simultaneous starts on different machines.

For directed path $P = (v_1, \dots, v_k)$, including a singleton, define

$$Q_{\delta}(P) = p_{v_k} + \sum_{h=1}^{k-1} \ell_{v_h v_{h+1}} \quad (6)$$

$$= \sum_{h=1}^k p_{v_h} - \sum_{h=1}^{k-1} \delta_{v_h v_{h+1}},$$

$$L_{\delta} = \max_P Q_{\delta}(P). \quad (7)$$

For release dates, define

$$Q_{r,\delta}(P) = r_{v_1} + Q_{\delta}(P), \quad L_{r,\delta} = \max_P Q_{r,\delta}(P). \quad (8)$$

Lemma 1 (Workload and path lower bounds). *With $W = \sum_i p_i$, every feasible schedule satisfies*

$$C_{\max}^{\text{OPT}} \geq \max \left\{ \frac{W}{m}, L_{r,\delta} \right\}. \quad (9)$$

When all releases are zero, the second term is L_{δ} .

Proof. The machines supply at most $mC_{\max}$ units of processing capacity, so $C_{\max} \geq W/m$. For any path $P = (v_1, \dots, v_k)$, repeated application of (2)–(3) gives

$$s_{v_k} + p_{v_k} \geq r_{v_1} + \sum_{h=1}^{k-1} \ell_{v_h v_{h+1}} + p_{v_k} = Q_{r,\delta}(P).$$

Maximizing over paths proves the claim. $\square$

A. BOS Priority

For every task i , define its overlap-adjusted bottom level by

$$b_i = \max \left\{ p_i, \max_{(i,j) \in E} (\ell_{ij} + b_j) \right\}, \quad (10)$$

where the inner maximum is omitted for a sink. The explicit singleton term p_i is necessary: with a large overlap allowance, a continuation can be shorter than task i itself.

Lemma 2 (Bottom-level dynamic program). *For every task i ,*

$$b_i = \max_{P: P \text{ starts at } i} Q_{\delta}(P). \quad (11)$$

Thus $\max_i b_i = L_{\delta}$, and all scores are computed in $O(|V| + |E|)$ time by a reverse topological scan.

Proof. For a sink, the only path starting at i is (i) , so both sides are p_i . Assume the statement holds for each successor j of i . A path beginning at i is either (i) or edge (i, j) followed by a path beginning at j . Its best length is therefore exactly the right-hand side of (10) by the induction hypothesis. Each vertex and edge is inspected once. $\square$

For a fixed offline instance, b_i is static. BOS is event-driven because the executable set changes, not because unchanged scores are recomputed. Online changes to G , p , r , or δ lie outside the model and guarantees in this paper.

III. EVENT-DRIVEN BOS

At time t , partition tasks into not-started, running, and completed sets $N(t)$, $A(t)$, and $C(t)$. After all events stamped t are closed, the executable set is

$$R(t) = \{i \in N(t) : t \geq r_i, j \in A(t) \cup C(t) \forall j \in \text{pred}(i), \\ t \geq s_j + \ell_{ji} \forall j \in \text{pred}(i)\}. \quad (12)$$

BOS chooses a task of maximum b_i from $R(t)$, breaking ties by a fixed task index. It is work-conserving: after equal-time events have been closed, an idle machine is never left unused while $R(t)$ is nonempty.

There are three event types: release at r_i , edge activation at $s_i + \ell_{ij}$, and completion at $s_i + p_i$. Starting a task can create zero-lag activations at the current timestamp. These events must be processed to closure before advancing time.

An implementation stores one release flag and an activated-incoming edge counter per task. It therefore updates only endpoints affected by an event rather than rescanning the DAG.

Theorem 3 (Feasibility and single execution). *Algorithm 1 starts every task exactly once and returns a schedule satisfying (2)–(4).*

Algorithm 1 Event-driven BOS

Require: DAG $G = (V, E)$, p , r , δ , and m **Ensure:** start times s_i and assignments μ_i

```

1: Compute  $\ell_{ij}$  and  $b_i$  in reverse topological order
2:  $N \leftarrow V$ ;  $A, C \leftarrow \emptyset$ ; initialize idle machines
3: Put every release event into a minimum-time event queue  $Q$ 
4: while  $C \neq V$  do
5:    $t \leftarrow$  smallest timestamp in  $Q$ 
6:   Process all release, activation, and completion events at  $t$ 
7:   Insert each newly executable task in a max-heap keyed by  $b_i$ 
8:   while an idle machine and an executable task both exist do
9:     Extract a maximum-key task  $i$  and assign an idle machine
10:    Set  $s_i \leftarrow t$ ; move  $i$  from  $N$  to  $A$ 
11:    Enqueue completion at  $t + p_i$ 
12:    Enqueue activation at  $t + \ell_{ij}$  for every  $(i, j) \in E$ 
13:    Close all newly generated events stamped  $t$  and update the heap
14:   end while
15: end while

```

Proof. A task enters the ready heap only while it is in N and is removed from N when started, so it cannot be started twice. Dispatch occurs only after the task's release and all incoming activation events, which proves (2) and (3). A task is assigned only to an idle machine, which remains occupied until its completion event, proving (4). Zero-lag closure can start a successor at the same timestamp as its predecessor, but only after the predecessor start has been recorded, so the argument still applies. $\square$

Lemma 4 (No deadlock and termination). *Algorithm 1 terminates after finitely many events.*

Proof. Every start removes one task from N , so there are at most $|V|$ starts. If $N \neq \emptyset$ and all machines are occupied, a running task has a finite next completion. Otherwise, take a source v in the induced DAG $G[N]$. Every predecessor of v has already started. If v is not executable, a finite release or activation event remains in the future; if neither does, v is executable and work conservation dispatches it. Equal-time events are exhausted before a strict time advance. There are $|V|$ releases, $|E|$ activations, and $|V|$ completions, excluding an infinite event sequence. $\square$

Proposition 5 (Implementation complexity). *With adjacency lists, counters, an event heap, and a ready heap, BOS runs in*

$$O((|V| + |E|) \log(|V| + |E|)) \quad (13)$$

time and $O(|V| + |E|)$ space.

Proof. Topological sorting and score computation are linear. Every task generates one release, start, and completion; every edge generates one activation. Each event and task enters a heap only a constant number of times. Graph storage, counters, state, and queues are linear. $\square$

The stated complexity excludes optional decision-trace snapshots. Saving and sorting every full ready set can require quadratic output size and additional sorting time. All timing

experiments disable these snapshots; feasibility validation and priority computation remain enabled.

IV. APPROXIMATION ANALYSIS

The following argument is deliberately policy-agnostic. It uses work conservation, but never the ordering induced by b_i . It therefore certifies BOS and every other work-conserving list rule under the same model; the particular BOS priority must be judged empirically.

For a completed schedule S , let $q(t)$ be the number of busy machines and define the non-full times and total idle machine-time as

$$\mathcal{T}_{\text{nf}} = \{t \in [0, C_{\max}) : q(t) < m\}, \quad (14)$$

$$I = \int_0^{C_{\max}} (m - q(t)) dt. \quad (15)$$

The integral is only accounting notation: the state is constant between a finite number of events.

For task v in the realized schedule, put

$$a(v) = \max(\{r_v\} \cup \{s_u + \ell_{uv} : (u, v) \in E\}). \quad (16)$$

Thus $a(v) \leq s_v$. Choose a last-finishing task v_k . Working backward from v_h , stop if r_{v_h} attains $a(v_h)$, selecting the release term in a tie. Otherwise choose a predecessor v_{h-1} satisfying

$$a(v_h) = s_{v_{h-1}} + \ell_{v_{h-1}v_h} \quad (17)$$

and continue. Acyclicity makes this process terminate and produces one directed path $P = (v_1, \dots, v_k)$. Let $\rho = r_{v_1}$ and $Q = Q_\delta(P)$.

Lemma 6 (Single blocking-chain cover). *Up to the measure-zero event instants, every non-full time belongs to*

$$\begin{aligned} \mathcal{T}_{\text{nf}} \subseteq [0, \rho) \cup \bigcup_{h=1}^{k-1} [s_{v_h}, s_{v_h} + \ell_{v_h v_{h+1}}) \\ \cup [s_{v_k}, s_{v_k} + p_{v_k}). \end{aligned} \quad (18)$$

The intervals after $[0, \rho)$ are pairwise disjoint, have total length Q , and each contains a running chain task. Consequently,

$$I \leq m\rho + (m-1)Q. \quad (19)$$

Proof. Take $t \in \mathcal{T}_{\text{nf}}$. If $t \geq s_{v_k}$, then t is before the makespan and lies in v_k 's execution interval. Otherwise $t < s_{v_k}$. If $t \geq a(v_k)$, task v_k would be executable while a machine is idle, contradicting work conservation; hence $t < a(v_k)$.

If the chain step into v_h uses predecessor v_{h-1} , then $t < a(v_h) = s_{v_{h-1}} + \ell_{v_{h-1}v_h}$. Either $t \geq s_{v_{h-1}}$, placing t in that predecessor prefix, or $t < s_{v_{h-1}}$ and the same argument moves one step backward. If the recursion reaches v_1 without covering t , work conservation gives $t < a(v_1) = r_{v_1} = \rho$. This proves (18).

For consecutive chain tasks, $s_{v_{h+1}} \geq s_{v_h} + \ell_{v_h v_{h+1}}$, so the charged prefixes are ordered and disjoint; the final execution interval starts after the last prefix ends. Their total length is $\sum_{h < k} \ell_{v_h v_{h+1}} + p_{v_k} = Q$. Because $\ell_{v_h v_{h+1}} \leq p_{v_h}$, task

v_h runs throughout its charged prefix. Thus at most $m - 1$ machines are idle on charged time after ρ , while at most m are idle before ρ . Integration proves (19). $\square$

Theorem 7 (Zero-release bound). *If every $r_i = 0$, any work-conserving list schedule S satisfies*

$$C_{\max}^S \leq \frac{W}{m} + \left(1 - \frac{1}{m}\right) L_\delta \leq \left(2 - \frac{1}{m}\right) C_{\max}^{\text{OPT}}. \quad (20)$$

Proof. Lemma 6 has $\rho = 0$ and $Q \leq L_\delta$, hence $I \leq (m - 1)L_\delta$. Capacity accounting gives $mC_{\max}^S = W + I$, which proves the first inequality. Both W/m and L_δ lower-bound the optimum by Lemma 1; applying the two bounds separately gives the ratio. $\square$

Theorem 8 (Release-aware bound). *For arbitrary finite releases, any work-conserving list schedule S satisfies*

$$C_{\max}^S \leq \frac{W}{m} + L_{r,\delta} \leq 2C_{\max}^{\text{OPT}}. \quad (21)$$

Proof. By capacity accounting and (19),

$$\begin{aligned} C_{\max}^S &= \frac{W + I}{m} \\ &\leq \frac{W}{m} + \rho + \left(1 - \frac{1}{m}\right) Q \\ &\leq \frac{W}{m} + \rho + Q. \end{aligned}$$

Because P starts at v_1 , $\rho + Q = Q_{r,\delta}(P) \leq L_{r,\delta}$. Lemma 1 then yields the factor two. $\square$

The release-free expression cannot simply be reused with releases: one unit-time task released at time R finishes at $R + 1$, whereas a formula containing only W/m and L_δ is independent of R . More broadly, the analysis assumes identical machines, non-preemption, a fixed DAG, finite releases, $0 \leq \ell_{ij} \leq p_i$, and work-conserving dispatch. It does not cover heterogeneous execution times, setup times, uncertain data, or online graph changes. Section V constructs a new fixed residual instance at a disruption; it does not apply this proof directly to interrupted original tasks.

V. CHECKPOINT-BASED RECOVERY

A. State and Physical Semantics

Let τ be a single disruption time in the nominal BOS schedule. All processing in $[0, \tau)$ is immutable. Its attained work is

$$x_i = \min\{p_i, \max\{0, \tau - s_i\}\}. \quad (22)$$

An operation with $0 < x_i < p_i$ may be interrupted exactly at τ ; completed work is retained without loss. For each remaining operation, a finite ordered set of additional checkpoints is specified. Thereafter preemption and migration are allowed only at these checkpoints. Every segment is non-preemptive and uses one of the same m machines. Checkpointing, suspension, and migration have zero overhead. No machine is lost or slowed in this model.

A material gate (i, g, a) means that operation i may execute up to attained work g , but cannot process beyond g before

absolute time a . We require $x_i \leq g < p_i$ and $a \geq \tau$ for newly introduced gates. Thus a delivery can block the second part of an operation while allowing its available-material prefix. A gate cannot retroactively invalidate already completed work. Gate times are known when recovery is planned; repeated unexpected revisions are outside the theorem.

A positive-lag handoff (i, j) requires attained work of at least ℓ_{ij} on i before the *first* start of j . Once satisfied, this handoff remains satisfied; it does not continuously stop and restart j when i pauses. A zero-lag edge requires that i has started, possibly simultaneously with j . All incoming handoffs use AND semantics. This is staged availability, not an arbitrary subset/OR assembly model.

B. Residual Graph Construction

For each unfinished task i , form sorted, distinct cut positions from x_i , p_i , remaining positive outgoing-lag thresholds, and material gates. Consecutive cuts $c_{i,h-1} < c_{i,h}$ define a segment $v = (i, h)$ of length $q_v = c_{i,h} - c_{i,h-1} > 0$. The residual time origin is τ . Add the following constraints.

- 1) Consecutive segments of one task have finish–start edges: $S_{i,h+1} \geq S_{i,h} + q_{i,h}$. This prevents parallel execution of multiple pieces of one original task.
- 2) For an unstarted successor j and an unsatisfied positive handoff (i, j) , let u be the segment ending at attained work ℓ_{ij} . Add a finish–start edge from u to the first segment of j . If $x_i \geq \ell_{ij}$, no residual edge is needed.
- 3) If $\ell_{ij} = 0$ and both tasks are unstarted, add a zero-lag start edge between their first segments. If i already started, that gate is satisfied. Incoming gates of a task that started before τ are already satisfied and are removed.
- 4) A segment beginning at material threshold g has release $\max\{0, a - \tau\}$. An unstarted task also retains its original release, shifted by τ and clipped at zero. Multiple release requirements on a segment are combined by their maximum.

A task scheduled to start at exactly τ is treated as unstarted; its pre-disruption execution prefix is empty. Simultaneous zero-lag starts are processed in topological order. Duplicate arc endpoints retain the strongest lag. Denote the resulting instance by G_τ , with N segments, M edges, releases R_v , and $D = \max_v R_v$.

Theorem 9 (Checkpoint transformation). *The residual graph is acyclic, preserves the remaining workload, and has $0 \leq \lambda_{uv} \leq q_u$ on every edge. Its feasible schedules are equivalent to feasible continuations under the specified checkpoint, material-gate, and progress-handoff semantics.*

Proof. Segments within a task are ordered by attained work. Between tasks, each edge respects the original DAG order. A lexicographic ordering by original topological position and within-task segment index therefore excludes cycles. Segment lengths telescope, giving $\sum_v q_v = \sum_i (p_i - x_i)$. Every edge has weight either zero or the full length of its tail segment, so its lag is bounded as required.

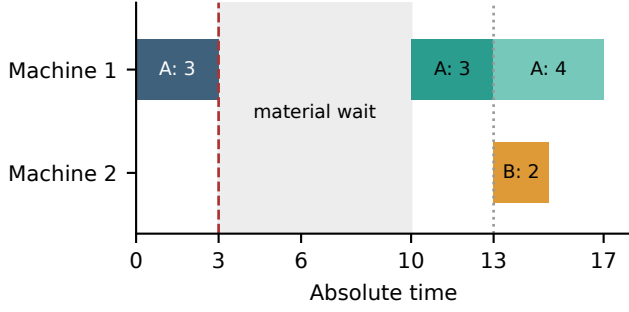


Fig. 1. Frozen work, material waiting, and progress-safe recovery in the worked example. The disruption occurs at time 3.

Given a residual schedule, concatenate the pieces of each original task in progress order, allowing idle gaps and machine changes. Within-task edges ensure no simultaneous processing of that task. An unsatisfied positive handoff is enforced by completion of the segment ending at its threshold. Satisfied handoffs are valid from the frozen prefix and remain valid because completed work is never lost. Zero-lag edges preserve the start condition. Segment releases impose material gates, and machine capacity is exactly that of the residual schedule. This produces a feasible physical continuation.

Conversely, split any legal checkpoint continuation into its prescribed segments. The same handoff and gate conditions imply every residual edge and release constraint, and non-overlapping machine intervals give capacity feasibility. The two representations have the same remaining makespan. This equivalence is only for these checkpoints and this frozen prefix; it does not cover arbitrary-time preemption or a different pre-disruption schedule. $\square$

There are at most $|V| + |E| + K$ segments when K material gates are added, because each distinct interior handoff threshold or gate introduces at most one cut. Sorting cut positions costs $O((|V| + |E| + K) \log(|V| + |E| + K))$ in a direct implementation. This bound is for a graph-oriented implementation; the supplied small-instance builder scans the original edge list per task for transparency.

C. A Consistent Interruption Example

Let $p_A = 10$, $p_B = 2$, $m = 2$, and $\ell_{AB} = 6$. Nominal BOS runs A on $[0, 10)$ and B on $[6, 8)$. At $\tau = 3$, a gate at attained work 3 blocks further work on A until time 10. The residual segments are A_1 of length 3, A_2 of length 4, and B of length 2. Their constraints are $R_{A_1} = 7$, $A_1 \rightarrow A_2$, and $A_1 \rightarrow B$, with both edges carrying lag 3. Thus A_1 runs on absolute times $[10, 13)$, A_2 on $[13, 17)$, and B on $[13, 15)$. The actual makespan is 17. Keeping the old elapsed-time test would incorrectly release B at time 6, when only three units of A have been completed. Figure 1 visualizes the corrected case.

VI. LP-LIST RECOVERY AND A PROTECTED PORTFOLIO

A. Relaxation and Exact Oracle

Assume integer segment lengths, lags, and release dates, as in the experiments. Let H be the smallest makespan returned by BOS, HEFT-rank, LPT, and FIFO on the residual instance. This feasible upper bound allows an optimal schedule to be restricted to $[0, H]$. For $T_v = \{R_v, \dots, H - q_v\}$, let y_{vt} indicate a start of segment v at time t , and let Z denote makespan. We solve

$$\min Z \quad (23)$$

$$\text{s.t.} \quad \sum_{t \in T_v} y_{vt} = 1 \quad \forall v, \quad (24)$$

$$\sum_v \sum_{\substack{s \in T_v: \\ s \leq t < s + q_v}} y_{vs} \leq m \quad t = 0, \dots, H - 1, \quad (25)$$

$$\sum_{t \in T_v} t y_{vt} - \sum_{t \in T_u} t y_{ut} \geq \lambda_{uv} \quad (u, v) \in E_\tau, \quad (26)$$

$$Z \geq \sum_{t \in T_v} t y_{vt} + q_v \quad \forall v, \quad (27)$$

$$Z \geq B_0, \quad 0 \leq y_{vt} \leq 1, \quad (28)$$

where $B_0 = \max\{\sum_v q_v / m, L_{R, \lambda}\}$ is the analytic residual lower bound. With binary y , this is our exact MILP oracle; with continuous y , its optimum is B_{LP} .

Proposition 10 (Oracle exactness and LP validity). *For integer data, the binary formulation attains the continuous-time non-preemptive segment optimum. Moreover, $B_0 \leq B_{LP} \leq \text{OPT}_{\text{rec}}$.*

Proof. Fix machine orders in a feasible continuous-time schedule. Release, precedence, and machine-order constraints are difference inequalities with integer weights. A cycle containing a machine-order edge would have positive weight and be infeasible; a cycle containing only original precedence edges is excluded by acyclicity. Earliest starts in this fixed-order graph are integer longest-path distances and do not delay any completion. Hence some optimum has integer starts.

Binary variables encode exactly these starts. Constraint (25) enforces maximum interval overlap at most m ; intervals can then be assigned greedily to m machines in start order. If no machine were available, more than m intervals would overlap. The remaining constraints enforce releases, lags, and makespan. Conversely every such integer schedule yields a feasible binary solution. Relaxing integrality enlarges the feasible region, and (28) explicitly retains B_0 , proving the bounds. $\square$

Fractional values describe a distribution of possible start positions, not simultaneous fractional processing of a physical task. The mean-lag constraints need not enforce precedence in every fractional sample. This weakness is allowed in a lower-bounding relaxation and is why independently rounding start times is unsafe.

Algorithm 2 Two-layer BOS with checkpoint recovery

Require: Nominal instance and an observed disruption at τ

- 1: Execute nominal BOS up to τ ; freeze completed progress
 - 2: Build G_τ with the known material gates and checkpoints
 - 3: Compute $S_B \leftarrow \text{BOS}(G_\tau)$ and $S_H \leftarrow \text{HEFT-rank}(G_\tau)$
 - 4: Solve the LP and obtain B_{LP} and $\bar{S}$
 - 5: **if** the LP succeeds **then**
 - 6: $S_L \leftarrow \text{executable-list dispatch ordered by } \bar{S}$
 - 7: $S^* \leftarrow \arg \min_{S \in \{S_B, S_H, S_L\}} C_{\max}(S)$
 - 8: **else**
 - 9: $S^* \leftarrow \arg \min_{S \in \{S_B, S_H\}} C_{\max}(S)$
 - 10: Use analytic lower bound B_0 for certification
 - 11: **end if**
 - 12: Validate the continuation and report the delay certificate
 - 13: **return** Frozen prefix followed by S^*
-

B. The Rounding Step

Define the fractional expected start $\bar{S}_v = \sum_{t \in T_v} ty_{vt}$. LP-list sorts priorities by increasing $\bar{S}_v$, breaking ties by segment index. It then runs the same event-driven dispatcher, selecting only currently executable segments. Thus the fractional start distributions become an integral priority order and then a feasible schedule. This is a *priority-based list rounding* rule; no independent rounding of $\bar{S}_v$ to a time is performed.

Feasibility and termination follow from the dispatcher proof. Because it never idles in the presence of an executable segment, LP-list also inherits the general list-scheduling bounds. No stronger approximation factor is claimed specifically from this LP, and interruption is not the rounding operation.

The protected portfolio cannot have a larger computed makespan than its BOS or HEFT-rank candidate. This is a property of selection, not empirical evidence of an LP advantage. Experiments therefore report LP-list alone and the two-greedy portfolio separately. LP model construction and solution time are charged to LP-list and the hybrid. Planning latency is measured but not inserted into simulated processing time; the theorem assumes a plan is available at the recovery epoch.

The LP contains $O(NH)$ variables and $O(N + M + H)$ constraints, with potentially many nonzeros in capacity rows. Its time index is pseudo-polynomial in numeric input values. The exact MILP is reserved for small instances. The greedy dispatch complexity does not imply that the complete LP-based method has the same complexity or scales to large horizons.

VII. MAXIMIZING TOLERABLE DELAY UNDER A DEADLINE

The makespan LP in Section VI answers a conditional question: *given the realized arrival delays, how should the remaining work be scheduled?* It does not directly maximize fault tolerance. Comparing this disturbed schedule with an undisturbed nominal BOS schedule would confound input

difficulty and algorithm quality. We therefore add a second, distinct optimization objective that reflects delivery slack.

Fix the same frozen prefix and a final promised deadline T . In residual time the horizon is $H_T = T - \tau$. Let $\mathcal{G}$ contain the segments that begin at designated material gates. For this controlled experiment, all gate materials have a common availability D after τ , and all other residual releases are zero. Define

$$D^*(T) = \max\{D \geq 0 : \text{a legal continuation finishes by } T\}. \quad (29)$$

The gate segments need not start at D ; it is their earliest allowed start. Thus increasing D measures how late the designated materials can arrive while preserving the promised deadline, not intentional idle time added to every operation. This is a scalar, common-arrival stress model, not a robust optimization result for arbitrary delay vectors.

Use the start variables of Section VI, now restricted to $t = 0, \dots, H_T - q_v$. Retain the assignment, capacity, and mean-lag constraints, remove the makespan minimization objective, and solve

$$\begin{aligned} \max \quad & D, \\ \text{s.t.} \quad & \sum_t ty_{vt} \geq D \quad v \in \mathcal{G}, \\ & 0 \leq D \leq H_T. \end{aligned} \quad (30)$$

Variable domains ensure that each integral segment finishes by the fixed horizon. The continuous relaxation gives U_D ; the binary version gives an exact tolerance oracle for integer data.

Proposition 11 (Tolerance bounds and verified rounding). *The relaxation satisfies $D^*(T) \leq U_D$. A feasible list-rounded continuation with gate delay $\hat{D}$ and makespan at most H_T certifies $\hat{D} \leq D^*(T)$.*

Proof. Every feasible integral schedule induces feasible start variables and satisfies (31); relaxing integrality can only increase a maximization optimum. Conversely, a validated rounded continuation is a witness in the feasible set of (29), so its delay cannot exceed the maximum. Integer starts suffice by the earlier fixed-order argument. For fixed integer starts the largest feasible common delay is the smallest gate start, an integer. Thus an optimum with integer D exists as well. $\square$

The distinction in bound direction matters: B_{LP} is a *lower bound on required completion time*, whereas U_D is an *upper bound on attainable delay tolerance*. Neither fractional solution alone certifies a physical execution.

Reserve-LP derives expected-start priorities from the solution of (30). For every integer $D \in [0, \lfloor U_D \rfloor]$, we apply these priorities to executable-list dispatch with the actual gate releases set to D , then validate deadline feasibility. The largest feasible tested value is reported. If a heuristic misses the deadline at every tested delay, the original fixed-order plan at $D = 0$ is retained as a validated fallback; such cases are counted explicitly. We scan all integer values instead of using binary search: although optimal feasibility is monotone in D , a

particular heuristic can exhibit scheduling anomalies. Reserve-hybrid selects the shortest of Reserve-LP, BOS, and HEFT-rank at each tested delay. Its feasible tolerance is at least that of each included candidate on this common scan.

All tolerance comparators use identical checkpoints, deadlines, and arrival gates. The fixed-plan repair comparator additionally retains the original machine assignment and original task order on each machine; release waiting is propagated through this fixed-order graph. It represents a conservative repair of the existing production plan, not work-conserving global redispatch. Random-32 includes 32 random priority lists plus BOS and HEFT-rank. Neither comparator is labelled an undisturbed nominal schedule.

This tolerance search is an offline design experiment whose full planning cost is recorded. It is not a constant-time recovery policy. All methods share the LP-derived scan limit for fairness of the tested delay range; reported total planning time includes that common LP solve as well as each method's complete scan. If the delay is uncertain but known to lie in $[0, \hat{D}]$, one may execute the validated plan for $\hat{D}$ without advancing any start when material arrives early. That fixed plan remains feasible for all smaller arrivals, avoiding an unsupported monotonicity assumption about re-running the heuristic.

VIII. A DIMENSIONALLY CONSISTENT DELAY CERTIFICATE

Let OPT_{rec} denote the minimum remaining makespan on G_τ with the *same* frozen prefix, material gates, and allowed checkpoints. It is not the optimum obtained by changing history. Write $W_\tau = \sum_v q_v$, $L_{R,\lambda}$ for its release-aware path bound, $D = \max_v R_v$, and $\alpha_m = 2 - 1/m$.

Theorem 12 (Sharpened release-aware upper bound). *Any work-conserving list schedule of G_τ , including LP-list and the selected hybrid, satisfies*

$$C_{\text{rec}} \leq \frac{W_\tau}{m} + \left(1 - \frac{1}{m}\right) L_{R,\lambda} + \frac{D}{m}, \quad (32)$$

$$C_{\text{rec}} \leq \min\{2\text{OPT}_{\text{rec}}, \alpha_m \text{OPT}_{\text{rec}} + D/m\}. \quad (33)$$

Proof. By Theorem 9, every residual arc satisfies the bounded-lag assumption. Apply the single blocking-chain lemma to the residual schedule, obtaining a path of lag-adjusted length Q rooted at release ρ . The capacity identity gives

$$\begin{aligned} C_{\text{rec}} &\leq W_\tau/m + \rho + (1 - 1/m)Q \\ &= W_\tau/m + (1 - 1/m)(\rho + Q) + \rho/m. \end{aligned}$$

Because $\rho + Q \leq L_{R,\lambda}$ and $\rho \leq D$, this proves (32). Both W_τ/m and $L_{R,\lambda}$ lower-bound OPT_{rec} , giving the second term in (33); the earlier release-aware theorem gives the first. The hybrid is one of these feasible list schedules, so the same inequalities apply to its selected candidate. $\square$

Corollary 13 (Sufficient ε -budget condition). *For any valid lower bound $0 < B \leq \text{OPT}_{\text{rec}}$, define*

$$\hat{\varepsilon} = \frac{D}{mB}. \quad (34)$$

Then

$$\frac{C_{\text{rec}}}{\text{OPT}_{\text{rec}}} \leq \min\{2, \alpha_m + \hat{\varepsilon}\}. \quad (35)$$

In particular, if $D \leq m\varepsilon B$, the requested $(2 - 1/m + \varepsilon)$ guarantee holds.

Proof. Divide (33) by OPT_{rec} and use $D/(m\text{OPT}_{\text{rec}}) \leq D/(mB)$. Substitute the sufficient budget condition. $\square$

D has units of time whereas ε is dimensionless. It is a normalized release budget, not an assumed LP rounding error. One can use the inexpensive B_0 , or the stronger B_{LP} when solved. For example, with $m = 2$, $B = 20$, and target $\varepsilon = 0.1$, the sufficient allowance is $D \leq 4$. Exceeding four does not prove failure; it means this sufficient condition no longer certifies the target. If a certificate independent of the realized releases is desired, use $B_{\text{free}} = \max\{W_\tau/m, L_\lambda\}$ on the same residual graph with all releases removed. Monotonicity gives $B_{\text{free}} \leq \text{OPT}_{\text{rec}}$, so $D \leq m\varepsilon B_{\text{free}}$ is a conservative advance budget.

Corollary 14 (Frozen-prefix end-to-end comparison). *If $C_{\text{rec}} \leq \gamma \text{OPT}_{\text{rec}}$ for $\gamma \geq 1$, then*

$$\frac{\tau + C_{\text{rec}}}{\tau + \text{OPT}_{\text{rec}}} \leq \gamma.$$

Proof. $\tau + C_{\text{rec}} \leq \tau + \gamma \text{OPT}_{\text{rec}} \leq \gamma(\tau + \text{OPT}_{\text{rec}})$ because $\tau \geq 0$. $\square$

There is no corresponding result here against an unrestricted clairvoyant optimum that can change the prefix or interrupt at arbitrary new points. Nor is the expression a universal online competitive ratio. For a large D , the factor-two bound remains valid under this model, but arbitrarily small ε cannot be claimed without checking its condition. Machine outages, restart losses, and positive switching costs break the present capacity or workload accounting and need their own model and proof.

IX. EXPERIMENTAL PROTOCOL

All reported experiments were newly run for this revision; none of the September 15 manuscript's aggregate counts is presented as a new measurement. Instances, schedules, solver statuses, seeds, raw CSVs, and table/figure generators accompany the manuscript.

A. Nominal Benchmarks

The full factorial design contains four DAG families (chain-heavy, fork-join, merge-heavy, and random-layered), $n \in \{24, 64, 128\}$, $m \in \{2, 4, 8\}$, target overlap fractions $a \in \{0, 0.25, 0.5, 0.75\}$, zero or uniform releases, and ten seeds $20260928 + s$, $s = 0, \dots, 9$. This gives 2,880 instances and 17,280 schedules for BOS, HEFT-rank, GRPW, LPT, SPT, and FIFO. All policies share releases, bounded lags, deterministic index tie-breaking, and the same event engine. GRPW scores the sum of a task's processing time and those of its distinct transitive successors. LPT and SPT use processing length; FIFO uses executable time.

Processing times are uniform integers in $[1, 20]$. For $0 < a < 1$, each overlap fraction is beta-distributed with mean a and concentration 14, then converted to an integer allowance; $a = 0$ gives zero allowance. Uniform releases range from zero to the rounded value of $0.5 \text{round}(W/m)$ (at least one). The code specifies every family construction. Chain-heavy instances are sanity cases, and zero-overlap instances check BOS/HEFT-rank agreement; both remain in the raw data. Comparative averages exclude these cases, leaving 1,620 pairs per comparator.

An independent small-instance suite uses the same four families, $n \in \{8, 12\}$, $m \in \{2, 3\}$, $a \in \{0.25, 0.75\}$, zero releases, and three seeds $280000 + s$. All 96 instances are sent to the binary time-indexed oracle. The exact table includes every family and reports optimum hits as well as mean and maximum gaps.

B. Matched Recovery and Tolerance Benchmarks

Recovery uses fork-join, merge-heavy, and random-layered instances, $n \in \{8, 12\}$, $m \in \{2, 3\}$, $a \in \{0.5, 0.8\}$, processing times in $[1, 10]$, zero nominal releases, and three seeds $928000 + s$. This creates 72 distinct frozen prefixes. The trigger is $\tau = \max\{1, \lfloor 0.35 C_{\text{nominal}} \rfloor\}$. All actively running tasks receive a gate at their current progress; each unstarted task is selected independently with probability 0.35 and receives a gate at $\lfloor p_i/2 \rfloor$. If none is selected, the first unfinished task is selected. Selection uses seed $61900 + s$.

The fixed-delay suite sets common relative gate arrival $D = \lceil dB_{\text{free}} \rceil$ for $d \in \{0, 0.25, 0.75\}$, yielding 216 scenarios with 6–24 residual segments. All recovery methods start from exactly the same prefix and residual graph. The zero-delay cases are rescheduling controls, not evidence that a physical delay occurred. Each scenario receives an exact residual makespan solve. Random-32 uses 32 sampled static priorities, with seed $424200 + \text{id}$, and also retains the two greedy candidates.

The tolerance suite uses the same 72 prefixes and deadlines $T = \lfloor (1 + s)C_{\text{nominal}} \rfloor$, $s \in \{0, 0.25, 0.5\}$. For each of the 216 prefix/deadline pairs, a tolerance LP and binary oracle are solved, then every integer delay through $\lfloor U_D \rfloor$ is scanned. Random priorities use seed $924200 + \text{base_id}$, held fixed across the delay scan. The zero-slack case means no extension of the nominal delivery promise; it does not force every algorithm to reproduce the original schedule.

C. Metrics, Validation, and Reproducibility

Nominal paired gain is $100(C_{\text{comparator}} - C_{\text{BOS}})/C_{\text{comparator}}$; positive values favor BOS. A makespan optimality gap is $100(C/\text{OPT} - 1)$. Tolerance is reported directly in time units, as $\hat{D}$, with its shortfall $D^* - \hat{D}$ and exact-hit count. These objectives are not pooled into a single ranking.

The nominal 95% intervals use 5,000 percentile bootstrap samples of ten shared-seed blocks, keeping all related design cells together. They are exploratory intervals for these generators, not industrial population estimates. Recovery and

TABLE I
NOMINAL PAIRED COMPARISONS (1,620 PAIRS PER ROW). POSITIVE GAINS FAVOR BOS; INTERVALS RESAMPLE TEN SHARED-SEED BLOCKS.

Comparator	Gain (%)	95% CI	W/T/L
HEFT-rank	0.205	[0.118, 0.284]	503/849/268
GRPW	0.310	[0.240, 0.381]	549/804/267
LPT	1.508	[1.342, 1.705]	1014/513/93
SPT	3.011	[2.837, 3.165]	1326/236/58
FIFO	1.941	[1.844, 2.039]	1239/312/69

tolerance have only three seed blocks, so their main tables are descriptive rather than claims of statistically established superiority.

Every candidate is checked for capacity, releases, hand-offs, and the applicable theoretical upper bound. Recovery validation additionally reconstructs original-task progress and machine intervals independently of residual edge tests. Exact optima require solver status zero and MIP gap at most 10^{-8} ; decoded integer schedules are validated before use. Every oracle has a 20-second limit, and failures would be retained rather than silently discarded. Seventeen regression tests cover the core scheduler, recovery semantics, tolerance oracle, and LP-failure fallback.

Experiments use Python, NumPy, and SciPy’s HiGHS interface [5] on a shared Linux x86-64 environment exposing AMD EPYC 9V74 CPUs. Exact package versions are recorded in `results/environment.json`. OpenBLAS and OpenMP environment thread counts are set to one. Runtime is wall time from one run per case and is environment-dependent. Greedy timings include priority calculation, dispatch, and core validation; LP timings include model construction, horizon computation, solution, and rounding. Physical reconstruction and the exact oracle are excluded from deployment-policy timings. Fixed-effort Random-32 is not advertised as an exactly equal-time comparison.

X. RESULTS

A. Nominal Quality and Exactness

All 17,280 main-suite schedules pass feasibility and applicable-bound checks. Table I shows paired gains on the nontrivial positive-overlap subset. BOS improves on HEFT-rank by only 0.205% on average, with 503 wins, 849 ties, and 268 losses. Its advantage is larger against simple processing-time priorities. These are generator-specific averages, not pointwise dominance.

All 96 nominal MILPs terminate with status zero and zero MIP gap. Table II shows that BOS has the lowest average gap among these six single priorities, but it is optimal on only 43 of 96 cases and its worst gap is 15.15%. This directly rejects an interpretation of BOS as an exact algorithm.

B. Fixed-Delay Recovery

All 216 residual makespan MILPs are solved to optimality; the largest reported MIP gap is 2.60×10^{-15} (floating-point tolerance). In 207 of these scenarios, at least one task is actively

TABLE II
NOMINAL EXACT SUITE: GAPS TO 96 CERTIFIED OPTIMA.

Policy	Mean gap (%)	Max (%)	Hits / 96
BOS	2.826	15.15	43 / 96
HEFT-rank	3.124	15.15	40 / 96
GRPW	3.361	18.75	42 / 96
LPT	3.467	20.00	41 / 96
SPT	5.642	34.09	35 / 96
FIFO	5.470	26.67	31 / 96

TABLE III
FIXED-DELAY RECOVERY, 216 CASES. GAPS ARE TO THE SAME RESIDUAL OPTIMUM; HITS COUNT EXACT MATCHES. TIMES EXCLUDE THE EXACT ORACLE.

Policy	Mean gap	Max gap	Hits	Mean ms
BOS	0.731%	11.11%	191	0.058
HEFT-rank	0.853%	11.11%	188	0.047
GRPW	0.848%	11.11%	187	0.052
LPT	5.498%	33.33%	109	0.042
SPT	10.634%	33.33%	30	0.041
FIFO	8.386%	50.00%	67	0.037
LP-list	4.745%	33.33%	105	3.273
Greedy-2	0.661%	11.11%	194	0.105
BOS-LP	0.500%	11.11%	200	3.378
Random-32	0.039%	8.33%	215	1.332

executing at the trigger; the remaining nine are valid between-operation controls. Table III compares matched residual inputs. “BOS” in this table means BOS priority on the checkpoint recovery DAG, not an undisturbed nominal schedule. Every policy receives the same delays and preemption options.

The continuous makespan LP has genuinely fractional variables in 211 of 216 cases. LP-list alone has a mean gap of 4.745%, compared with 0.731% for BOS-recovery and 0.661% for Greedy-2. Against Greedy-2, LP-list wins six cases, ties 110, and loses 100. Selecting the best of the three candidates reduces the mean gap to 0.500%, with 200 exact hits. Its average paired makespan gain over Greedy-2 is 0.151%. Figure 2 emphasizes the difference between a weak standalone LP ordering and the protected selection result.

The conclusion is not caused solely by zero-delay controls. For $d = 0.25$, mean gaps are 1.055% for BOS-recovery and 6.047% for LP-list; at $d = 0.75$, they are 0.382% and 2.607%, respectively. Large imposed waiting can make priority differences less important relative to the final makespan. These comparisons do not test the separate tolerance-maximization objective.

Random-32 attains 215 of 216 exact makespans with mean gap 0.039%. Its mean measured time is 1.332 ms versus 3.378 ms for BOS-LP and 0.105 ms for Greedy-2. LP-list never beats Random-32 on this suite. Since Random-32 uses 32 random schedules rather than one schedule, and the experiment does not impose identical wall-time budgets, this is a fixed-effort quality/cost comparison rather than a controlled equal-time superiority theorem. It nevertheless shows that the small LP benefit cannot be assumed to be the best use of extra computation.

The analytic ε certificate is at most 0.1 in 105 of 216

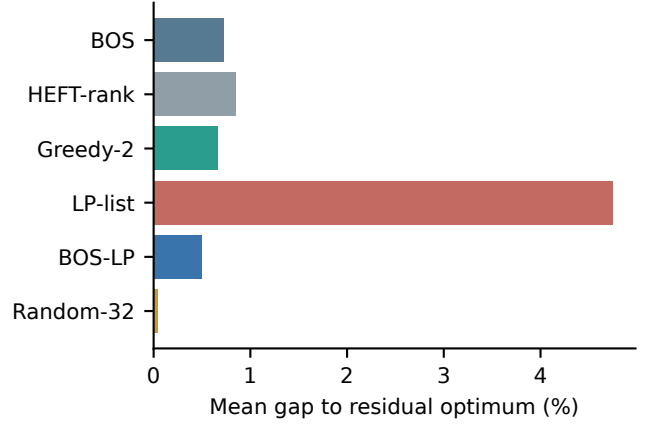


Fig. 2. Mean residual makespan gap on matched fixed-delay cases. BOS-LP is the protected BOS/HEFT-rank/LP-list portfolio.

TABLE IV
DELAY TOLERANCE UNDER MATCHED DEADLINES (216 CASES). SHORTFALL IS MEAN $D^* - \hat{D}$ IN TIME UNITS; HITS ARE EXACT. FALLBACK COUNTS USE THE FEASIBLE ZERO-DELAY FIXED PLAN.

Policy	Mean $\hat{D}$	Shortfall	Hits	Fallback
Fixed-plan	5.963	5.898	12	0
BOS-recovery	11.616	0.245	194	2
Greedy-2	11.634	0.227	198	2
Reserve-LP	10.569	1.292	89	22
Reserve-hybrid	11.801	0.060	204	0
Random-32	11.852	0.009	214	0

cases: all 72 zero-delay controls and 33 mild-delay cases. The makespan LP strictly strengthens the analytic lower bound in only nine cases and does not increase this certificate count. Thus lower-bound certification is a valid LP role, but this particular relaxation is not empirically strong on these data. All applicable sharp-bound and physical-progress checks pass.

C. Fixed-Deadline Delay Tolerance

Table IV evaluates the distinct objective requested in Section VII. All 216 tolerance MILPs achieve status zero with zero MIP gap. The LP upper bound equals the integer optimum in 100 cases; in the other cases, feasible list rounding is needed to turn an upper limit into a deliverable schedule. All reported delay values have validated schedule witnesses.

The existing fixed-order plan tolerates a mean delay of 5.963 time units. Greedy-2 recovery increases this to 11.634, and Reserve-hybrid to 11.801. The corresponding mean shortfalls from the exact tolerance are 5.898, 0.227, and 0.060. Thus reorganizing the remaining work and allowing checkpoint migration has substantial value against conservative fixed-plan repair. Most of this gain is already present with greedy redispatch and cannot be attributed to the LP alone.

Reserve-LP alone averages 10.569 units and requires a zero-delay fixed-plan fallback in 22 cases where its list ordering misses even the zero-delay deadline. Reserve-hybrid needs no such fallback, achieves 204 exact tolerances, and has maximum shortfall two. Random-32 achieves 214 exact tolerances

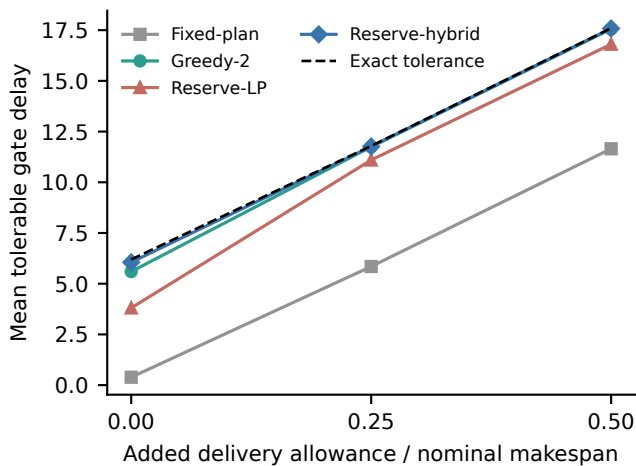


Fig. 3. Maximum validated common gate delay versus extra delivery allowance. Each point averages the same 72 frozen prefixes.

and maximum shortfall one; its mean tolerance is 11.852. The full tolerance scan takes a mean 6.938 ms for Reserve-hybrid and 24.493 ms for Random-32 in this run, including the shared LP used to bound the scan. These measurements suggest a quality/cost tradeoff; they do not establish that one method dominates for large instances or realistic disruption distributions.

Figure 3 shows the effect of the delivery allowance. A larger promised completion window permits larger material delay for all methods. This is the intended meaning of tolerance slack; it must not be mistaken for an unexplained degradation of makespan.

XI. DISCUSSION AND LIMITATIONS

The analysis isolates three different quantities: nominal overlap allowance δ_{ij} , residual material waiting D , and dimensionless approximation allowance ε . They are not interchangeable. A delivery allowance $T - C_{\text{nominal}}$ is a fourth quantity: it defines a feasibility target, while the tolerance LP asks how late materials may arrive before that target becomes impossible.

The $(2 - 1/m + \varepsilon)$ statement is conditional on an explicit budget test and uses a frozen-prefix residual optimum. The proof does not exploit BOS’s particular priority or the LP objective. Consequently, it certifies the scheduling framework but does not prove that either priority is better than other work-conserving choices. The empirical comparison includes losing cases and a stronger fixed-effort comparator to keep this distinction visible.

Several limitations constrain interpretation. All inputs are synthetic, and the recovery graphs have at most 24 segments. Only three recovery seed blocks are used; larger and independently sampled disruption suites are needed. No public industrial trace or native RCPSP/max benchmark is evaluated. All machines are identical and continuously available; checkpointing and migration are free. Physical operations that

cannot be interrupted, setup-dependent machines, spoilage, rework, finite buffers, and transport delays require additional constraints.

The gate model permits progress before a missing material is needed, but it does not model arbitrary alternative-component choices. It also assumes all future gate arrival times are known at the recovery epoch. An adaptive policy facing new information repeatedly would need a reveal model and a different performance benchmark. Planning time is measured, not simulated as production downtime; using these policies in real time requires accounting for that latency. The time-indexed LP is pseudo-polynomial and no large-scale scalability claim is made for it.

XII. CONCLUSION

The checkpoint recovery model preserves completed work and enforces progress-safe handoffs. Single-chain accounting yields a release-aware bound and a checkable $(2 - 1/m + \varepsilon)$ delay condition. Distinguishing fixed-delay makespan from fixed-deadline tolerance avoids conflating speed with resilience. Experiments support rescheduling but do not establish LP priorities as the strongest heuristic. The supplied model, proofs, exact oracles, and reproducible code provide an auditable basis for larger benchmark and industrial evaluations.

REFERENCES

- [1] R. L. Graham, “Bounds on multiprocessing timing anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, Mar. 1969.
- [2] M. Bartusch, R. H. Möhring, and F. J. Radermacher, “Scheduling project networks with resource constraints and time windows,” *Annals of Operations Research*, vol. 16, pp. 199–240, 1988.
- [3] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, Mar. 2002.
- [4] A. Schutt, T. Feydy, P. J. Stuckey, and M. G. Wallace, “Solving RCPSP/max by lazy clause generation,” *Journal of Scheduling*, vol. 16, no. 3, pp. 273–289, 2013.
- [5] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.